Benchmarks Aren’t Enough

Measuring and diagnosing AI agents in the wild — a five-paper reader’s guide

🤖 Agentic Engineering Papers · catch-up digest, 5 June 2026

June 5, 2026

Chapter 0: The Leaderboard Is Lying to You

GPT-5.5 solves 84.4% of an English browsing benchmark and 45.67% of the same task in Korean. A dozen frontier models look strong on generic tool benchmarks and not one clears 50% the moment the tools reach into a personal account. The best planner finishes two-thirds of household tasks when the rules are stated up front and stumbles the instant they’re revealed one violation at a time. Every model in a compiler-construction study scores a perfect 100 on stage one and not one finishes stage six. And nearly 37% of the agent runs that produced a correct final answer still contain a harmful error buried somewhere in the trajectory.

Five research teams, working on five different problems, arrived independently at the same conviction: **a single clean number on a leaderboard systematically overstates what an agent can actually do.** That is the spine of this bundle. The five papers aren’t a stack of unrelated benchmark releases — they’re four coordinated pushes against the same failure of measurement, and reading them together is the point.

The core illusion they share a name for is *transfer that never happened*. A model that aces a benchmark looks like it has a general capability. What it often has is a benchmark-shaped one wearing a general one's clothes. The four threads below are the four seams where the costume comes apart.

The locale seam — competence that doesn't cross a boundary. K-BrowseComp moves a browsing agent from the English web into Korean sources and watches the score crater; the models built *specifically* for Korean did worst of all, because fluency in a language is not competence at acting inside its information environment. MCP-Persona moves the agent in a different direction — out of the open web and into the apps people live in, Slack and Lark and Reddit, where the tools touch individual accounts and real state. Same lesson, different axis: the capability that looked general was scoped to the English, stateless, public slice of the world the benchmark happened to sample. As you read these two, watch for *where the score craters* — it names the slice of the world your own evals are silently failing to sample.

The time seam — failures that only exist over turns. AdaPlanBench withholds the rules and discloses them only when the agent's plan violates them, and the most diagnostic failure isn't a bad first plan — it's the agent re-breaking a rule it was *already told about*, the equivalent of an employee who twice suggests the thing you just ruled out. A static benchmark, where everything is stated up front, can't produce that failure at all. Watch for the gap between a valid plan and a finished task: models clear 90% on producing executable-looking plans and stay under 45% on actually succeeding.

The horizon seam — capability that decays as the job gets long. RAMP chains six engineering stages so each consumes the last one's output, and completion slides from 100% to 20% with no model finishing the pipeline — not because the late tasks are harder, but because early defects compound and context exhausts. Its second, quieter warning is cost: a 2,525x spend spread between models doing identical work, which means the model that wins on quality can lose by three orders of magnitude on the bill. Watch for both — the decay curve *and* the price tag, because isolated single-task benchmarks hide them equally.

The resolution seam — knowing *where*, not just *whether*. TELBench and its DRIFT method stop scoring the final answer and start auditing the trajectory, hunting the first unsupported claim that actually changed the outcome. Their sharpest data point reframes everything before it: outcome correctness and process correctness are genuinely different properties, and 36.9% of *successful* runs still hide a harmful error. Watch for the distinction between exploration (supposed to be weakly supported) and commitment (where the agent actually goes wrong) — the discipline that turns a verdict into a diagnosis.

The unifying engineering message is blunt and worth carrying into every chapter: **trust the evals that match the shape of your deployment** — its locale, its statefulness, its progressive disclosure, its horizon length, its need for process-level error attribution — and distrust any single leaderboard number that flattens all of that into one figure. Each paper also hands you a method you can lift

without reproducing the benchmark: a backwards adversarial question generator, a tool-traversal simulator, a constraint-withholding harness, a resurrection protocol, a claim ledger.

Start where the costume first slips — in Chapter 1, where the browser learns to speak Korean and a frontier model’s retrieval skill quietly fails to make the trip.

Chapter 1: When the Browser Speaks Korean

Paper: K-BrowseComp: A Web Browsing Agent Benchmark Grounded in Korean Contexts · arXiv 2606.02404 · **Code:** github.com/prometheus-eval/K-BrowseComp

GPT-5.5 can solve 84.4% of the problems on BrowseComp, the English-language gauntlet built to break web-browsing agents. Hand it the same kind of problem grounded in Korean web sources — a question about an album's fourth track, a poem buried in a thirteenth slot of a poet's tenth collection — and it solves 45.67%. DeepSeek-V4-Pro falls from 83.4% to 30.00%. The retrieval skill these models supposedly have does not survive the trip across a language boundary.

That is the finding worth carrying out of this paper, and it has a sharp edge for anyone shipping agents outside the English-speaking web: your browse evals are quietly lying to you. A model that aces an English benchmark has not demonstrated that it can act as a competent web agent. It has demonstrated that it can act as a competent web agent *on the English web*, where the page structures, search conventions, entity names, and indexed sources all happen to match what it saw in training. Move the same task into Korean and the score craters. The competence didn't transfer because there wasn't a general competence to transfer in the first place — there was a locale-specific one wearing a general one's clothes.

A benchmark built to resist easy answers

K-BrowseComp is 400 problems split two ways. The core is a 300-problem verified subset, hand-written by 17 native Korean annotators under strict rules: every question must be grounded in Korean context, hard to answer by direct search but easy to verify once you've found the answer, and require at least four reasoning steps or constraints — either multi-hop (one finding leads to the next search) or parallel-branching (intersect several independent constraints to pin down one answer). Annotators were forbidden from using LLMs to write questions, barred from paywalled or non-textual sources, and required to produce a single answer that stays stable over time. Every item was then re-verified by the authors against public web evidence.

The questions are deliberately gnarly. One asks for the title of an OST Part 4 song containing the word "sun," from a 2016–2017 webtoon-based drama that held the second-highest peak rating among that broadcaster's same-day dramas. To answer it you have to fix the right drama *before* you go looking for lyrics — and the paper shows models reliably skipping that step, anchoring on the wrong drama, and confidently returning a song from it.

Korea’s own models do worst

Here is the part that should give any “sovereign AI” program pause. The models released through Korea’s government-funded Proprietary AI Foundation Model program — built specifically for Korean — scored *lowest of all*: K-EXAONE-236B at 10.33%, A.X-4.0 at 5.33%, HyperCLOVAX-SEED-Think-32B at 2.33%, and Kanana-2 at a flat 0.00%, because it couldn’t emit tool calls that satisfied the harness at all. Speaking the language fluently bought them nothing on the task of *acting* in the language’s information environment.

The trajectory analysis explains why, and it’s the second reusable insight here. The failures mostly aren’t retrieval failures. Models find relevant Korean evidence and then lose the thread: they commit to a candidate before verifying every constraint, run parallel searches but never intersect the results into a single filtered candidate set, or bind an intermediate finding to the wrong role when the entity type shifts mid-question. Tellingly, wrong answers used *more* search calls than right ones across nearly every model — they kept searching and still failed. Korean-targeted post-training improved fluency without improving the long-horizon **state maintenance** that browsing actually demands. Parameter count didn’t substitute for it either.

Turning the problem-solving asymmetry into a problem generator

The remaining 100 problems are where the method gets interesting. Browsing tasks have a built-in asymmetry: solving them is hard, but verifying a found answer is comparatively easy. The authors flipped that to ask whether *creating* hard problems is also hard — and found that, done naively, it is. Off-the-shelf “write me a hard question” prompts produced problems that were either trivially solvable or ill-defined.

Two interventions fixed it. They fed the generator hard human-written problems as few-shot exemplars, and they pointed it at a taxonomy of eight failure modes mined from watching models fail on the verified set — things like semi-structured parsing failures and constraint-tracking failures. They used Claude Code (claude-opus-4.7 at maximum effort) as the proposer: it opens a real Korean web page and constructs a question *backwards* from it, withholding the answer, the source URL, and the page’s most identifying entity, so that reaching it again requires genuine multi-hop or parallel retrieval. Each candidate then ran a three-filter gauntlet — too-searchable questions got rewritten, the answer had to be uniquely extractable from the source page, and the item was kept only if every target model failed. The yield was 37.3%; on the surviving 100 synthetic problems the strongest model managed just 26.00%.

What you can actually do with this

If you're putting an agent into a non-English market, the concrete move is to stop trusting English browse scores as a proxy and build or borrow a locale-grounded eval before launch. You don't need 300 hand-written items to learn something — even a few dozen questions written by native speakers, grounded in the local web's actual page structures and entity-naming conventions, will tell you whether your agent retrieves *and holds onto* evidence in that environment, or just retrieves it and forgets the constraints.

The generation trick is the part most teams can lift directly, and it isn't Korea-specific. Watch your agent fail on a small seed set, label the recurring failure modes, then hand those modes plus the source pages to a strong model and have it construct adversarial questions backwards from real pages — filtering out anything a baseline solver can already answer. The create-versus-solve asymmetry means you can scale a hard, self-verifying eval far past what hand-authoring would allow, in any language or domain where answers are publicly checkable. And the failure taxonomy is worth keeping even if you never generate a synthetic question: candidate capture, unmerged branches, misbound chains, and shaky answer finalization are a ready-made checklist for diagnosing *why* your own agent is wrong, not just that it is.

That checklist points at the same uncomfortable truth from a different angle — that an agent which looks capable in a clean benchmark can come apart the moment it touches a messier, more personal slice of the real world. The next chapter pushes on exactly that seam. Where K-BrowseComp moved the agent into an unfamiliar *language*, MCP-Persona moves it into unfamiliar *stakes*: a benchmark that drops LLM agents into simulated versions of the apps people actually live in — Reddit, Xiaohongshu, Lark, Slack — where the tools don't query the open web but reach into individual accounts and local databases, and getting it wrong means acting on someone's real personal data.

Chapter 2: Agents Meet Your Actual Accounts

Paper: MCP-Persona: Benchmarking LLM Agents on Real-World Personal Applications via Environment Simulation · arXiv 2606.02470 · **Code:** github.com/wwh0411/MCP-Persona

A user tells their agent: “Send a polite message to my supervisor, Song Ke, explaining my condition and requesting leave.” The environment holds Song Ke’s Lark user ID. The agent never goes looking for it. It fires off a WeCom message to a recipient it invented, then declares the task done. On the surface the output reads fine — polite, on topic. It’s also completely wrong: wrong platform, wrong person, no grounding. That single trace is this benchmark in miniature, and it’s the gap between agents that look competent and agents you’d actually let touch your accounts.

The benchmark nobody had built yet

The **Model Context Protocol** is now everywhere — Apple and Alibaba wiring LLMs into mobile assistants, Anthropic’s Skills and the OpenClaw ecosystem letting anyone bolt custom tools onto an agent. But the benchmarks meant to measure how well agents *use* those tools have lagged. Almost all of them — MCP-Universe, ToolAthlon, and kin — center on **generic, information-seeking tools**: search a map, query a database, fetch a web page. Stateless. No login. Nothing that belongs to *you*.

That leaves a blind spot exactly where the highest-value MCP use lives. The tools people actually want agents to drive — Reddit, Xiaohongshu (Rednote), Lark/Feishu, Slack, Notion, Instagram — are **personal**. They touch individual accounts and local databases. They’re stateful: creating, modifying, and deleting real entities. And they’re impossible to benchmark openly, because doing it for real means handing over private data and account credentials.

MCP-Persona is the first benchmark built for this personal, account-scoped slice: 24 MCP servers (12 of them personalized) and 173 human-verified tasks across four scenario families — collaboration platforms, content management, social media, and email — including cross-server tasks that force an agent to coordinate tools across multiple apps.

How they tested it without leaking your data

The clever bit is the simulation. You can’t give a benchmark live access to real Slack workspaces, so the authors built faithful local replicas. Their **Tool-Traverse** method doesn’t just read each tool’s docs — it probes the real server with both valid and deliberately broken inputs (type mismatches, missing fields, out-of-range values, contradictory combos), records how the server responds to each, then has an LLM write executable Python that reproduces that behavior, error codes and all. State

persists across turns via a generated context handler that does the load/save/query/modify work of a real backend.

The payoff: the simulated tools behave almost indistinguishably from the live ones. On 50 Lark traces, Tool-Traverse hit **94% behavioral accuracy and 93.8 F1**, versus 53.3 F1 for a documentation-only baseline, and roughly tripled response-similarity scores. The doc-only version emitted generic error templates; the traversal-built version reproduced the server's actual nested response structure. That fidelity is what makes the agent results trustworthy rather than artifacts of a sloppy mock.

The headline: competent on paper, lost in your accounts

Now the uncomfortable part. Across more than a dozen state-of-the-art models — GPT-5, the Claude 4.x line, Gemini, Grok, the Qwen and DeepSeek families — **none cleared 50% accuracy**. The best, Claude-Sonnet-4.5, landed at 38.66% overall accuracy and 41.5% execution accuracy; GPT-5 trailed at 36.99%. And the metric that matters most for real use — Success Rate at 0.8, the share of tasks an agent nails end-to-end — topped out at a brutal **10.4%**. Most tasks, the agent gets *partway* and then derails.

These are the same models that look strong on generic tool benchmarks. Point them at personal, stateful, account-scoped tools and competence evaporates. The trajectory analysis surfaces three failure modes worth memorizing. First, **under-exploration**: when a detail is implied rather than stated, weaker models guess a plausible action instead of querying the environment for the truth. Second, **skipping dependent steps**: a task needs the agent to resolve a coworker's internal ID from their phone number before scheduling a meeting — and the agent just jams the phone number into the ID field or fabricates one. Third, **long-context decay**: as turns stack up and document tools dump huge payloads, models lose track of the constraints they were given at the start.

What this means if you're shipping MCP integrations

The blunt implication: **generic tool-use evals overstate your readiness for personal tools.** If your roadmap has an agent acting on real user accounts — posting, messaging, scheduling, editing documents — a strong public-leaderboard score tells you almost nothing about how it'll behave against stateful, login-bound operations. The capability gap is large and it's concentrated in exactly the stuff that touches user state.

So, concretely, three things you can borrow. **Simulate before you ship.** Copy the core move: stand up a sandboxed replica of your tool — fake accounts, local state — and probe it with both valid and malformed calls so you capture the real error envelopes, not just the happy path. Test destructive operations there, not in production. **Audit for the three failure modes by name.** Write evals that check whether your agent resolves implicit information instead of guessing, executes latent dependency steps (resolve-then-act chains), and holds early constraints after a long document

dump. **Trim the tool menu and lean on good skill docs.** Two ablations point the same way: cutting candidate tools to just the ones a task needs improved accuracy, especially in long contexts, and task-aligned skill docs helped further — hand-refined guides beat the most-downloaded public OpenClaw skills, which were sometimes stale or setup-focused. And the paper found no correlation between spend and performance (GPT-5 hit its accuracy at roughly $0.09 per task), so optimize the accuracy-cost trade-off, not raw model price.

MCP-Persona reveals what breaks when the world an agent operates in is hidden inside accounts it has to go digging through. The next chapter pushes on a different kind of hidden world. In AdaPlanBench the constraints aren't buried in a database for the agent to query but withheld entirely — revealed only after the agent proposes a plan that violates them, forcing it to re-plan against a world that pushes back one rejection at a time.

Chapter 3: Planning When the Rules Show Up Late

Paper: AdaPlanBench: Evaluating Adaptive Planning in Large Language Model Agents under World and User Constraints · arXiv 2606.05622 · **Code:** github.com/JiayuJeff/AdaPlanBench

Your volleyball got wet and is now too slippery to play with. How do you dry it? An agent answers: use a towel. Reasonable. Except there's no towel at home. So it pivots: use a fan. Also reasonable, except the user hates fans for drying things because they're slow, and would rather not, thank you. Neither of those facts was in the original prompt. They only surfaced *because the agent proposed a plan that tripped over them.*

That little drama is the entire point of **AdaPlanBench**, a benchmark from the University of Illinois Urbana-Champaign built to measure something most agent evaluations skip: not whether a model can plan, but whether it can keep re-planning as the rules of the game get revealed to it one violation at a time. And the headline finding is sobering. Across ten leading models, the best — GPT-5 — finishes only **67.75%** of tasks. Open-weight models mostly sit at or below 30%.

Two kinds of constraints, both hidden

The authors argue that real-world planning runs into two flavors of constraint, and both are usually unspoken at the start. **World constraints** are physical: there's no iron in the house, the sink is broken, the mop is missing. **User constraints** are preferences: I dislike disposable items, I won't wait for a fan to dry something slowly. Existing benchmarks tend to test one or the other. AdaPlanBench insists on both at once, because that's the situation any household assistant — or any agent embedded in a messy real workflow — actually lives in.

The clever part is the runtime. AdaPlanBench is built on 307 curated household tasks, each augmented through an automated pipeline with a dual-constraint profile. At evaluation time those constraints are **withheld**. The agent proposes a plan; LLM judges check it against the hidden world and user constraints plus a quality rubric; and only the constraints the plan actually *violates* get disclosed back as feedback. The agent must then infer what it's missing, track what it's already learned, and revise — turn after turn, under an accumulating pile of rules it discovers only by breaking them. That loop is what separates "can plan once" from "can adapt."

Why a valid plan isn't a finished task

The numbers underneath the headline are where the engineering lessons hide. Most models keep a **valid-plan rate** above 70% — they generally produce executable-looking plans. Gemini-3.1-Pro and

Gemini-3-Flash both clear 90% on that metric. Yet their actual task accuracy stays below 45%. A plan that doesn't crash is not a plan that succeeds.

Worse, models keep re-breaking rules they've already been told about. Averaged across the ten models, each task sees 0.295 repeated violations of *already-disclosed* world constraints and 0.503 repeated violations of disclosed *user* constraints — roughly double. Nearly 18% of tasks end in early stopping, where the agent spins for two turns violating known constraints without surfacing a new one — the agentic equivalent of an employee who twice suggests the thing you just told them won't work.

Performance also degrades on two axes at once. The more constraints a task carries overall — the benchmark grades tasks into low, mid, and high tiers — the worse models do. And *within* a single task, plan quality drops turn over turn as disclosed constraints pile up. Models can hold a coherent plan at the start and lose the thread as the requirement set grows.

The user-constraint problem

The sharpest finding for anyone building agents: **user constraints hurt more than world constraints.** In an ablation isolating each source, "user-only" tasks were consistently harder than "world-only" tasks, and the dual setting was hardest of all. The authors' read is that a single user preference quietly rules out a *wide swath* of the action space — "no disposable items" eliminates dozens of plausible tools at once — even though it reads as just one line. Physical limits tend to block one specific move; preferences block whole strategies.

And the obvious fixes underdeliver. The team tried re-injecting every previously disclosed constraint into the prompt each turn — a memory crutch. It improved validity (VPR up 5–15%) but barely moved accuracy (under 3% for three of four models). They tried feeding back rubric scores on failed dimensions; accuracy crept up ~10% while VPR *collapsed* by up to 40%, because models fixated on the newest complaint and broke things they'd already gotten right — a recency bias that trades one failure for another. When failures were dissected, two rubric dimensions stayed stubbornly weak: **effectiveness** (would this actually accomplish the goal?) and **physical plausibility** (does this tool use make physical sense?). Raw scale didn't rescue any of it — Qwen3 at 8B, 14B, and 32B performed about the same, and GPT-5-Mini nearly matched GPT-5.

What you could try with this

The first thing to take away is a gut-check on your own agents. If you've validated a planner by giving it a fully-specified task and watching it produce a good plan, you've tested the easy 30% of the problem. Borrow AdaPlanBench's move: deliberately **withhold constraints and reveal them only when the agent's proposal violates them.** Even a hand-built harness — a checker that surfaces one hidden rule per failed attempt — will tell you fast whether your agent adapts or just confidently

repeats itself. Watch specifically for re-violations of rules it's already been told, the single most diagnostic failure here.

The second is where to invest. The user-constraint weakness points straight at preference-tracking and memory across turns as a high-leverage place to spend effort — *but* note the paper's warning that naively dumping all prior constraints back into context wasn't enough. The gap isn't just retention; it's globally consistent revision that honors old preferences while fixing new problems. If your agent touches user preferences that surface mid-task, treat that as a distinct, hard capability to build and test for, not a freebie that comes with a bigger model.

The trouble in AdaPlanBench is rules arriving late inside a single conversation. The next chapter widens the lens to a different kind of accumulation — RAMP, a production-grounded runtime assessment of long-horizon software-engineering agents, where the failure isn't a hidden preference but the sheer length of the job: task completion collapses across the serial stages of a real engineering pipeline, sliding from 100% at the first stage to 20% by the last, with no model managing to carry a task all the way through.

Chapter 4: What Breaks When the Workflow Gets Long

Paper: Benchmarks are Not Enough: RAMP for Runtime Assessing of Agentic Models in Production Systems · arXiv 2605.27492 · **Code:** github.com/arcsysu/YatCC

Every model in this study aced the first task. All fifteen of them — frontier flagships, mainstream workhorses, lightweight budget options — scored a perfect 100 on setting up the build environment. Then they walked up a six-stage compiler-construction pipeline, and by the final stage only one in five managed to finish the job. Not one model completed the whole thing. The headline that survives all the tables is brutal in its simplicity: capability that looks flawless on an isolated task degrades dramatically the moment you chain tasks together.

That gap is the whole point of **RAMP** (Runtime Assessment of Models in Production). The authors argue that the way we currently grade agents — static, isolated, short-horizon benchmarks where the agent's state resets after every task — systematically flatters them. Real production engineering doesn't reset. It carries forward a repository, intermediate artifacts, a modified environment, accumulated context, and the consequences of every earlier decision. RAMP is built to measure exactly that carry-forward.

A workload that actually depends on itself

RAMP runs on the YatCC platform, using a real compiler-construction project as its workload. The six stages are genuinely serial: environment setup, then a lexer that tokenizes C source, a parser that builds an AST, IR generation, IR optimization, and finally RV64 assembly generation. Each stage consumes the artifact the previous stage produced. A broken token stream poisons the parser; a malformed AST poisons everything downstream. This is the structural feature almost every coding benchmark lacks — SWE-bench, ProgramBench, and friends start each task from a clean repository and end with a single patch, so they never see what happens when early defects compound.

The clever piece of engineering here is the **resurrection** protocol. Serial dependencies create a measurement problem: if a model fails at the parser, you can't tell whether it would have nailed IR generation, because it never got a valid AST to work with. So when a task score drops below the passing threshold, RAMP silently swaps in a "golden" reference artifact and lets the agent continue from the next stage. The agent isn't told this happened. That one trick separates *cannot reach* from *cannot solve* — it lets you score downstream capability even when the upstream pipeline is broken.

The collapse, in numbers

Run without resurrection — the pure end-to-end autonomy test — the completion rates tell the story stage by stage: 100% at setup, 46.7% at the lexer, 26.7% at the parser, 13.3% at IR generation, 0% at IR optimization, and 20% at the final assembly stage. The trend isn't even clean, which is itself the finding: the last stage scores *higher* than the second-to-last, so this isn't "the tasks just got harder." It's the cumulative damage of earlier edits to a shared environment and the quiet propagation of subtle errors. Opus-4.7 led the leaderboard with a mean reward of 93.39 and still stalled at IR optimization. GPT-5.5 nailed the first four stages perfectly, then emitted completely non-functional backend code. The average agent landed only 17.6 points above a do-nothing baseline.

When the team dug into *why* agents died, they found two dominant failure modes that isolated benchmarks would never surface. **Context exhaustion** was the single most common hard stop — nine of fifteen models ran out of context window as accumulated code, dialogue history, and instructions piled up, mostly at the parser and IR stages. The other was premature strategic abandonment: agents that explicitly *decided* to skip hard stages to conserve budget, redefining the task rather than doing it.

Cost is not a footnote

Here's the part that should change how you pick a model. Total inference cost across these fifteen runs ranged from $0.05 to $126.24 — a **2,525x spread**, roughly three orders of magnitude, between models doing the same work. Opus-4.7's chart-topping reward cost a 14.5x premium over the second-place DS-v4-Pro for a 9.4% improvement. Qwen-3.6-Max ranked fifth on quality but cost 19.6x more than DS-v4-Flash, which beat it.

To make this legible, RAMP introduces the **Agent Efficiency Index** (AEI), which equally weights pipeline depth, reward, time, cost, and token usage. Under AEI the rankings invert: Opus-4.7's raw-reward crown drops it to an AEI of 40.00 because it maxed out every resource axis, while GPT-5.5 — a lesser model by reward — tops the efficiency table at 81.57. Same task, and the "best" model by one metric is the worst by another.

What you could try

The transferable lesson isn't about compilers. It's that **isolated benchmark scores are a weak predictor of production behavior**, and you can audit for the gap cheaply. If your agent does real multi-step work, build at least one evaluation where tasks are genuinely serial — where stage two consumes stage one's output and a stage-one defect must propagate. Watching completion fall across that chain tells you more than any single-task pass rate. Borrow the resurrection idea too: inject a known-good intermediate state and re-run from there, so you can tell whether a late failure is a real capability ceiling or just inherited garbage. And before you commit to a model on a leaderboard

tie, measure cost, latency, and token burn on *your* workflow — a model that matches another on quality can run a thousand times more expensive, and that only shows up when you instrument the process, not the answer.

RAMP scores the trajectory holistically: how deep an agent got, what it spent, where it broke. But it still treats each stage as pass-or-fail and tells you the *category* of failure, not the precise moment things went wrong. The next chapter pushes that resolution further. TELBench and its DRIFT method tackle span-level error localization — pinpointing which specific segment of a deep-research agent's trajectory introduced the flaw that made the final answer unreliable, rather than only scoring whether the answer was right.

Chapter 5: Finding the Exact Step That Went Wrong

Paper: Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories · arXiv 2606.02060 · **Code:** github.com/NJU-LINK/DRIFT

A deep-research agent fails the way a sloppy detective fails. It doesn't usually botch the final accusation. It accepts a shaky lead an hour earlier, treats that lead as fact, and builds everything afterward on top of it. By the time the wrong name comes out of its mouth, the actual mistake is buried twenty steps back, wearing the costume of an established fact.

That gap is the whole problem this paper goes after. Scoring an agent on its final answer tells you **that** it failed. It tells you nothing about **where**. And for a system that solves tasks through long trajectories of search, tool calls, evidence inspection, and synthesis, "where" is the only thing you can actually fix.

The first wrong step, not the visibly wrong answer

The researchers reframe reliability as a property of the process, not the outcome. Their data makes the case better than any argument could: across their annotated corpus, 97.3% of failed trajectories contain at least one harmful error span, but so do 36.9% of *successful* ones. Agents reach the right answer despite unsupported intermediate commitments, and they fail in ways the final answer can't explain. Outcome correctness and process correctness are related but genuinely different things.

To study the process, they collect **2,790 real agent trajectories** — two agent frameworks (MiroFlow and OAgent), three backbone models (GPT-5, Gemini-2.5-Pro, Claude-Sonnet-4.5), three benchmarks (GAIA, XBench, BrowseComp). Raw logs are too long and framework-specific to compare, so each trajectory is converted into ordered **semantic spans**: coherent chunks of work like planning, retrieval, verification, or finalization. Then two LLM annotators from different model families propose high-recall candidate errors, and seven human experts — over 300 hours each — verify every candidate against the trajectory's own evidence, revising and adjudicating.

The output is **TELBench**, a 1,000-instance benchmark. The hard part isn't spotting obvious mistakes; it's distinguishing a genuinely harmful error from all the benign noise that surrounds it — normal exploration, failed searches it recovered from, tentative hypotheses, tool noise. A span counts as an error only when the agent introduces, relies on, amplifies, or finalizes an unsupported or contradicted judgment that actually shapes the answer. Average trajectory: about 12 spans. Some of those spans look suspicious and are harmless; some look routine and are fatal.

Audit the claims, not the spans

The naive approach is to hand an LLM the whole trajectory and ask it to find the errors. It's unstable: it mistakes benign exploration for a mistake, over-focuses on the final answer, or misses the early unsupported commitment that quietly poisoned everything downstream.

So the authors built **DRIFT**, and its core move is the most portable idea in the paper. Instead of scoring spans in isolation, DRIFT audits the *claims* an agent makes. A **Claim Keeper** reads the full trajectory and builds a ledger: every decision-relevant belief, where it was introduced, where it first became consequential, and which later spans reuse it. This is what separates an exploratory candidate name in a search query from that same name treated as a settled premise. A **Support Seeker** then grades each consequential claim against the trajectory's evidence — direct, weak, missing, or conflicting. Crucially, weak support alone doesn't make a span an error. A final **Dependency Tracer** decides which under-supported claims were actually committed to and propagated into the answer path, and flags only those.

The results are decisive. DRIFT beats bare full-trajectory prompting and beats generic agentic auditors (Codex, Claude Code wrapped as auditors) on every backbone, improving span-level error localization and first-error accuracy by **up to 30 percentage points**. The honest part: simply wrapping an LLM in a more elaborate agent workflow doesn't help — Codex and Claude Code gave inconsistent gains and sometimes made things worse. The structure mattered, not the agent-iness. Two more findings worth keeping: scaling the backbone up doesn't reliably help, and even DRIFT struggles to pinpoint the *earliest* error in long, hard trajectories. Detecting that something went wrong and naming the first decisive wrong step are different skills, and the second is still genuinely hard.

What you could build from this

The actionable core needs no new model and no benchmark. It's a way of thinking about your own deep-research or search agents.

Instrument your agent to keep a claim ledger. As it runs, log each decision-relevant claim — an entity it selected, a constraint it accepted, an evidence reading, a "no answer exists" conclusion — along with the step it appeared in and the steps that later depend on it. Most agent frameworks already emit enough trace data to reconstruct this after the fact.

Then audit support, but gate on consequence. For each claim that later reasoning actually relies on, check whether the trajectory contains direct evidence, weak evidence, no evidence, or contradicting evidence. Don't flag everything weakly supported — exploration is supposed to be weakly supported. Flag the first unsupported claim that *changes the answer*, then trace forward to the spans that inherited it. That single discipline — hunting the earliest consequential unsupported commitment instead of re-reading the final answer — is the paper's whole contribution distilled into a check you can run today.

It pairs naturally with where errors actually concentrate. The paper found retrieval is high-volume but low-risk (only 2.9% of retrieval spans are errors), while decision-making and finalization carry normalized error rates of 60.5% and 51.8%. If you're adding human review or automated guardrails, spend them where the agent *commits*, not where it *searches*. The search step rarely lies; the moment it decides what the search meant is where it goes wrong.

This is the note the whole evaluation theme of this bundle ends on. The earlier chapters pushed past clean leaderboards toward harder, more honest measurement; this one pushes past the final answer entirely. Reliability isn't a score you read off the output — it's a property of the path the agent took to get there, and that path is auditable. You can find the first decisive wrong step. Once you can do that, evaluation stops being a verdict you deliver after the fact and becomes a process you can instrument, inspect, and improve while the agent is still working — which is the only kind of reliability that ever actually compounds.